

CPT102-2526 Exercise 11

Solving Graph Problems - Soln

In Lecture 11, we have learned how to find **connected components** in an undirected graph, a **topological order** in a DAG, and **strongly connected components** in a directed graph. Use them to solve the following problems.

You may write your algorithm in either pseudocode or Java. Then, justify its correctness and explain how it satisfies the required worst-case running time.

Problem 11.1 Counting Rooms

You are given an $n \times m$ grid representing a building. Each cell is either floor `.` or wall `#`. You can move up, down, left, and right through floor cells. Count how many separate rooms exist. The required worst-case running time is $O(n \times m)$.

Hint: Describe how to build a graph and then use the algorithm to find connected components.

Solution:

Treat the grid as an undirected graph. Each floor cell `.` is a vertex. Two vertices are adjacent if their cells share a side (up, down, left, or right). A room is exactly one connected component of this graph: all cells in the same room are mutually reachable through floor cells, while cells in different rooms are not connected. We do not need to explicitly store this graph. Instead, scan the grid and start a DFS/BFS whenever an unvisited floor cell is found. That search marks exactly all cells in one room, so the number of searches started is the number of rooms.

```
COUNT-ROOMS(grid, n, m):
    visited[0..n-1][0..m-1] <- all false
    rooms <- 0
    directions <- {(-1,0), (1,0), (0,-1), (0,1)}
    for r <- 0 to n-1 do
        for c <- 0 to m-1 do
            if grid[r][c] = "." and not visited[r][c] then
                rooms <- rooms + 1
                DFS(r, c)
    return rooms
```

```
DFS(r, c):
    visited[r][c] <- true
    for each (dr, dc) in directions do
```

```

nr <- r + dr; nc <- c + dc
if 0 <= nr < n and 0 <= nc < m then
  if grid[nr][nc] = "." and not visited[nr][nc] then
    DFS(nr, nc)

```

The outer loops inspect all $n \times m$ cells once. Each floor cell is marked at most once by DFS, and when a cell is processed, only four neighbours are checked. Therefore the total number of neighbour checks is $O(n \times m)$, and thus the worst-case running time is $O(n \times m)$.

Problem 11.2 Hamiltonian Path in DAG

Find a Hamiltonian path in a general graph is an open problem, but we can do it easily for a DAG. Given a DAG, write an algorithm to determine whether there is a directed path that visits each vertex exactly once. The required worst-case running time is $O(E + V)$.

Hint: Use a topological sort.

Solution:

Compute a topological ordering of the DAG, written as $\text{order}[1], \text{order}[2], \dots, \text{order}[|V|]$. If there is a directed edge $\text{order}[i] \rightarrow \text{order}[i+1]$ for every consecutive pair in this order, then these edges form a directed path that visits every vertex exactly once, so the DAG has a Hamiltonian path. If some consecutive pair has no such edge, then no Hamiltonian path exists. The reason is that any Hamiltonian path in a DAG forces its vertices to appear in that same order in every topological ordering: the path edges force the first path vertex before the second, the second before the third, and so on.

```

DAG-HAMILTONIAN-PATH( $G = (V, E)$ )
  order <- TOPOLOGICAL-SORT( $G$ )
  for  $i <- 1$  to  $|V|$  do
    position[order[i]] <- i
  hasNext[1..|V|-1] <- all false
  for each directed edge  $(u, w)$  in  $E$  do
    if position[w] = position[u] + 1 then
      hasNext[position[u]] <- true
  for  $i <- 1$  to  $|V|-1$  do
    if hasNext[i] = false then
      return false
  return true, order

```

If the algorithm returns true, then for every i from 1 to $|V|-1$ the edge $\text{order}[i] \rightarrow \text{order}[i+1]$ exists. These $|V|-1$ edges concatenate to form a directed path through all vertices in the topological order, and no vertex repeats, so this is a Hamiltonian path. Conversely, suppose the DAG has a Hamiltonian path $p[1] \rightarrow p[2] \rightarrow \dots \rightarrow p[|V|]$. Since each path edge is a directed edge of the DAG, every topological order must place $p[1]$ before $p[2]$ before $\dots$ before $p[|V|]$. Thus the computed topological order is exactly this path order, and every consecutive pair in it must be connected by the corresponding path edge. Therefore the algorithm returns true.

Topological sorting takes $O(V + E)$ time. Computing positions takes $O(V)$, scanning all edges takes $O(E)$, and checking hasNext takes $O(V)$. Thus the total worst-case running time is $O(V + E)$.

Problem 11.3 Odd-length Directed Cycle

Given a directed graph, write an algorithm to determine whether it has an odd-length directed cycle. The required worst-case running time is $O(E + V)$.

Hint: Use the algorithm to find strongly-connected components.

Solution:

First compute the strongly connected components (SCCs) of the directed graph. Then, for each SCC, ignore the directions of the edges inside that SCC and test the resulting undirected graph for bipartiteness using BFS/DFS two-colouring. Return true if any SCC is non-bipartite; otherwise return false.

A directed cycle is always contained inside one SCC, because every vertex on the cycle can reach every other vertex on the cycle by following the cycle. Therefore, if G has an odd-length directed cycle, then one SCC contains that cycle. After directions are ignored, the same vertices and edges form an odd undirected cycle, so that SCC is non-bipartite.

Conversely, consider one strongly connected component C . If C has no odd directed cycle, then its underlying undirected graph must be bipartite. To see this, choose any root s in C and colour each vertex by the parity of the length of a directed path from s to that vertex. This colour is well-defined: if two s -to- v paths had different parity, then combining them with a directed path from v back to s would create an odd closed directed walk, which contains an odd directed cycle. Also, every directed edge $u \rightarrow v$ must go between opposite colours; otherwise the path from s to u followed by $u \rightarrow v$ would give an s -to- v path of the wrong parity. Hence the underlying undirected graph is bipartite. The contrapositive says that if the underlying graph of an SCC is non-bipartite, then that SCC must contain an odd directed cycle.

Run an SCC algorithm such as Kosaraju. For each SCC, run a standard bipartite test on the subgraph induced by that SCC after treating every directed edge $u \rightarrow v$ as an undirected edge $\{u, v\}$. If the test ever finds an edge whose endpoints already have the same colour (including a self-loop), return true. If all SCCs pass the bipartite test, return false.

Finding all SCCs takes $O(V + E)$ time. The bipartite tests over all SCCs also take $O(V + E)$ total time, because each vertex is coloured once and each edge is examined only when considering the component containing both endpoints. Therefore the overall worst-case running time is $O(V + E)$.

Problem 11.4 (Optional) Correctness of Kosaraju Algorithm

In a DFS of a directed graph G where marked vertices are considered in a topological order (reverse postorder) given by a DFS of the reverse graph G^R (Kosaraju's algorithm), the vertices reached in each call of the recursive method from the constructor are in a strong component.

Hint: First, prove by contradiction that every vertex v that is strongly connected to s is reached by the call $\text{dfs}(G, s)$ in the constructor. Second, prove that every vertex v reached by the call $\text{dfs}(G, s)$ in the constructor is strongly connected to s .

Solution:

Kosaraju's algorithm first runs DFS on the reverse graph G^R and records vertices in reverse postorder. It then considers vertices in that order and, whenever it sees an unmarked vertex s , runs $\text{dfs}(G, s)$. We prove that each such DFS call marks exactly the strong component containing s .

First, every vertex strongly connected to s is reached by $\text{dfs}(G, s)$. Let v be strongly connected to s , and suppose for contradiction that v is not reached during $\text{dfs}(G, s)$. Since s can reach v in G , the only reason $\text{dfs}(G, s)$ would not reach v is that v was already marked before this call began. But v can also reach s . Therefore, the earlier DFS call that marked v would also have been able to reach and mark s . In that case s would already be marked, and the constructor would not start a new call $\text{dfs}(G, s)$, a contradiction. Hence every vertex in the strong component of s is reached.

Second, every vertex reached by $\text{dfs}(G, s)$ is strongly connected to s . Let v be any vertex reached during $\text{dfs}(G, s)$. Then there is a directed path $s \rightarrow v$ in G . It remains to show that $v \rightarrow s$ in G , or equivalently that $s \rightarrow v$ in the reverse graph G^R .

Suppose, for contradiction, that s cannot reach v in G^R . Since $s \rightarrow v$ in G , there is a path $v \rightarrow s$ in G^R . Thus, in G^R , v can reach s but s cannot reach v . In any DFS of G^R , this implies that v finishes after s : if the DFS starts from v first, it reaches s before finishing v ; if the

DFS starts from s first, it cannot reach v , so s finishes before a later DFS eventually visits v . Therefore v appears before s in the reverse postorder used by the second pass.

But, the second pass considers vertices in that reverse postorder. If v appears before s , then v would either have started an earlier $\text{dfs}(G, v)$ call or would already have been marked before s was considered. This contradicts the fact that v is first reached during $\text{dfs}(G, s)$. Hence s must reach v in G^R , which means v reaches s in G . Since we already know s reaches v in G , v is strongly connected to s .

Therefore, the call $\text{dfs}(G, s)$ reaches all vertices strongly connected to s and no vertices outside that strong component. Therefore each DFS call in the second pass of Kosaraju's algorithm outputs exactly one strongly connected component.

This is the end of CPT102-2526 Exercise 11 **Soln**.